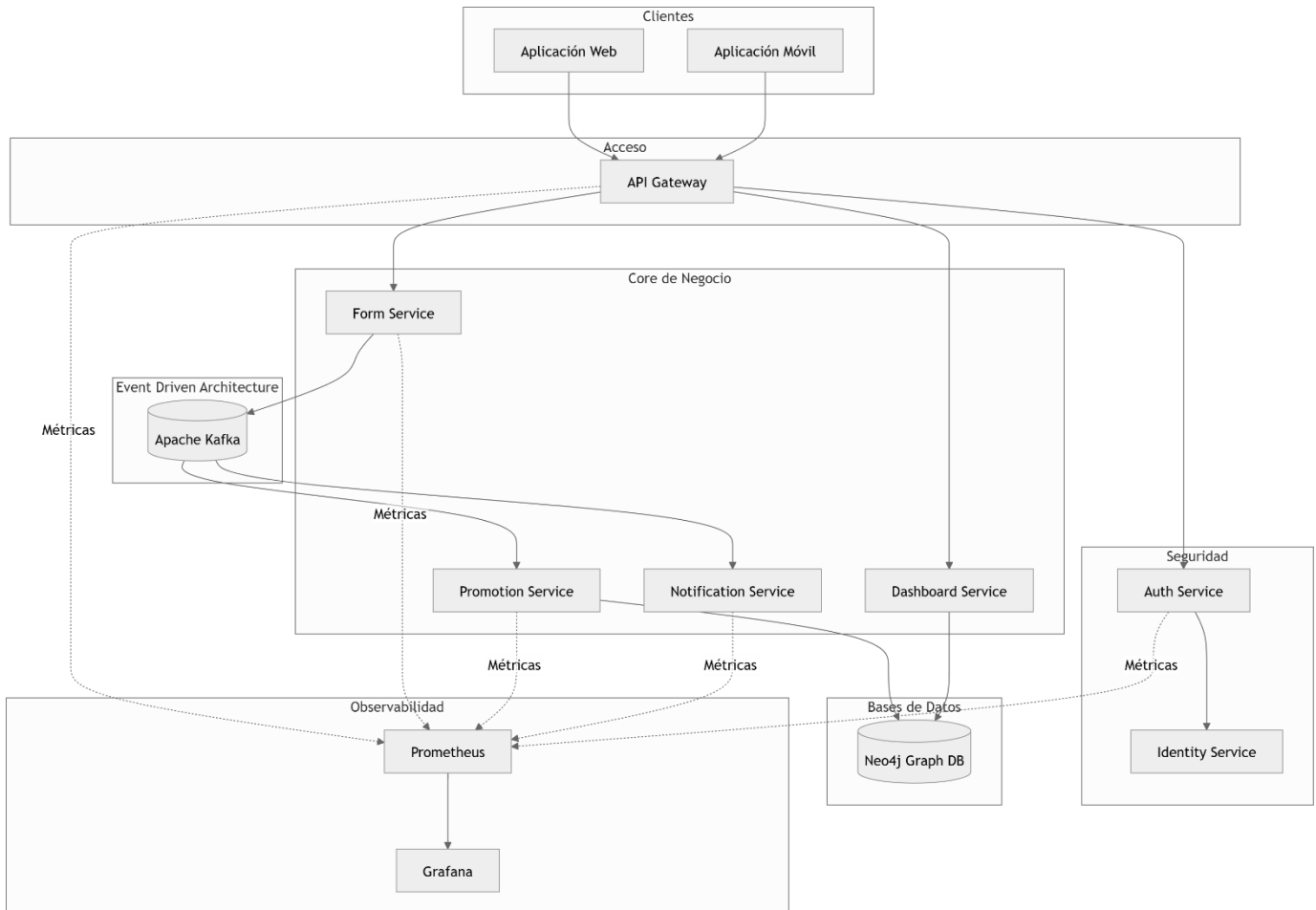


Documentación Completa del proyecto

1. Arquitectura detallada con diagramas



Descripción de la Arquitectura

La arquitectura de CircleGuard está basada en microservicios y se encuentra organizada en capas funcionales que permiten separar responsabilidades, facilitar el mantenimiento y mejorar la escalabilidad del sistema.

En la capa de clientes se encuentran la aplicación web y la aplicación móvil, las cuales representan los puntos de acceso de los usuarios al sistema.

Todas las solicitudes ingresan a través del API Gateway, que actúa como punto único de entrada. Este componente es responsable de enrutar las peticiones hacia los microservicios correspondientes y centralizar aspectos relacionados con acceso y comunicación entre servicios.

La capa de seguridad está compuesta por Auth Service e Identity Service. Auth Service gestiona los procesos de autenticación y autorización, mientras que Identity Service se encarga de la gestión y anonimización de identidades para proteger la información sensible de los usuarios.

El núcleo de negocio está conformado por Form Service, Promotion Service, Notification Service y Dashboard Service. Estos servicios implementan la lógica principal de la aplicación y se comunican tanto de forma síncrona mediante APIs REST como de forma asíncrona mediante eventos publicados en Apache Kafka.

La comunicación basada en eventos permite desacoplar los servicios, mejorando la escalabilidad y la tolerancia a fallos del sistema. Los eventos generados por Form Service son consumidos por Promotion Service y Notification Service para procesar información y ejecutar acciones derivadas.

La persistencia de relaciones y propagación de información se realiza mediante Neo4j Graph Database, utilizada especialmente por Promotion Service para almacenar y consultar estructuras basadas en grafos.

Finalmente, la capa de observabilidad está compuesta por Prometheus y Grafana. Prometheus recopila métricas de los diferentes servicios, mientras que Grafana permite visualizar dichas métricas mediante dashboards para monitoreo y análisis del estado de la plataforma.

Flujo de Funcionamiento del Sistema

El funcionamiento general de CircleGuard sigue el siguiente flujo:

1. El usuario accede a la plataforma desde la aplicación web o móvil.
2. La solicitud es enviada al API Gateway, que actúa como punto central de entrada al sistema.
3. Cuando se requiere autenticación, el Gateway redirige la solicitud al Auth Service, el cual valida las credenciales y consulta al Identity Service cuando es necesario obtener o verificar información de identidad.
4. Una vez autenticado el usuario, puede interactuar con las funcionalidades del sistema, como el diligenciamiento de formularios mediante Form Service.
5. Form Service procesa la información recibida y publica eventos en Apache Kafka para notificar a otros componentes de la plataforma.
6. Promotion Service consume los eventos publicados en Kafka y actualiza la información almacenada en Neo4j, permitiendo analizar relaciones y procesos de propagación.
7. Notification Service consume los mismos eventos para generar notificaciones dirigidas a los usuarios mediante los canales configurados.

8. Dashboard Service consulta la información consolidada y genera indicadores, estadísticas y visualizaciones para los administradores del sistema.
9. Paralelamente, Prometheus recolecta métricas de los microservicios y las almacena para su análisis.
10. Grafana consulta las métricas almacenadas por Prometheus y genera dashboards de monitoreo que permiten supervisar el estado, rendimiento y disponibilidad de la plataforma en tiempo real.

Este flujo permite mantener una arquitectura desacoplada, escalable y orientada a eventos, facilitando la integración de nuevos servicios y el crecimiento futuro de la solución.

2. Descripción de la metodología ágil implementada

Enfoque Metodológico

El desarrollo de CircleGuard se realizó utilizando un enfoque ágil e incremental, orientado a la implementación progresiva de funcionalidades y a la integración continua de los diferentes componentes de la arquitectura. Debido a la naturaleza académica del proyecto, el trabajo se organizó en etapas de desarrollo enfocadas en objetivos específicos, permitiendo validar cada componente antes de avanzar a la siguiente fase.

La estrategia consistió en construir inicialmente los microservicios principales y posteriormente incorporar herramientas de despliegue, automatización, monitoreo y observabilidad. Este enfoque facilitó identificar errores de integración de manera temprana y realizar ajustes continuos durante el desarrollo.

Organización del Trabajo

Las actividades se gestionaron mediante un conjunto de tareas priorizadas según las necesidades del proyecto. Cada tarea fue desarrollada, probada e integrada de manera progresiva, permitiendo mantener una versión funcional del sistema durante las diferentes etapas de implementación.

Las principales actividades fueron:

- Implementación de la arquitectura de microservicios.
- Configuración de autenticación y autorización.
- Integración mediante eventos utilizando Kafka.
- Contenerización de servicios mediante Docker.
- Despliegue de componentes en Kubernetes.
- Automatización de procesos mediante Jenkins.
- Configuración de monitoreo con Prometheus.

- Implementación de infraestructura como código utilizando Terraform.
- Ejecución de pruebas unitarias e integración.

3. Documentación de patrones de diseño

Repository Pattern

La capa de persistencia se encuentra desacoplada de la lógica de negocio mediante repositorios basados en Spring Data. Este patrón permite abstraer el acceso a datos y facilita el mantenimiento del código.

Beneficios:

- Separación de responsabilidades.
- Mayor facilidad para pruebas unitarias.
- Menor acoplamiento con la tecnología de persistencia.
- Service Layer Pattern

La lógica de negocio se encuentra encapsulada en servicios especializados como **HealthStatusService**, **IdentityVaultService** y **JwtTokenService**.

Beneficios:

- Centralización de reglas de negocio.
- Reutilización de funcionalidades.
- Mejor organización del código.

Controller Pattern

Cada microservicio expone sus funcionalidades mediante controladores REST que reciben solicitudes HTTP y delegan el procesamiento a la capa de servicios.

Beneficios:

- Separación entre presentación y lógica de negocio.
- Mayor mantenibilidad.
- Facilidad para exponer APIs REST.
- API Gateway Pattern

El acceso a los microservicios se centraliza mediante el Gateway Service, encargado de enrutar solicitudes, aplicar validaciones y controlar el acceso a la plataforma.

Beneficios:

- Punto único de entrada.
- Simplificación de la comunicación cliente-servidor.
- Mejor control de seguridad y monitoreo.

Event-Driven Architecture

La comunicación entre algunos componentes se realiza de manera asíncrona mediante Apache Kafka. Los eventos generados por Form Service son consumidos por servicios como Promotion Service y Notification Service.

Beneficios:

- Desacoplamiento entre servicios.
- Mayor escalabilidad.
- Mejor tolerancia a fallos.

Patrones de Resiliencia y Configuración

Circuit Breaker

Implementado mediante la clase SimpleCircuitBreaker y utilizado por IdentityClient dentro del Auth Service.

Propósito:

Evitar que una falla temporal del servicio Identity afecte completamente el funcionamiento del servicio de autenticación.

Beneficios:

- Prevención de fallos en cascada.
- Recuperación controlada.
- Mayor disponibilidad del sistema.

External Configuration

La configuración de los servicios se gestiona mediante archivos de propiedades y variables de entorno.

Ejemplos:

- circleguard.identity-service.url
- circleguard.features.identity-fallback
- Configuración JWT

- Credenciales de bases de datos
- Variables utilizadas en Kubernetes

Beneficios:

- Separación entre código y configuración.
- Facilidad para desplegar en múltiples ambientes.
- Mayor seguridad y mantenibilidad.

Feature Toggle

Se utiliza la propiedad `circleguard.features.identity-fallback` para habilitar o deshabilitar comportamientos alternativos cuando el servicio de identidad no se encuentra disponible.

Beneficios:

- Activación o desactivación de funcionalidades sin modificar código.
- Reducción del riesgo durante despliegues.
- Mayor flexibilidad operativa.

Estilo Arquitectónico

La solución implementa una arquitectura de microservicios apoyada en eventos y contenedores.

Las principales características son:

- Servicios independientes y desacoplados.
- Comunicación síncrona mediante APIs REST.
- Comunicación asíncrona mediante Kafka.
- Despliegue en contenedores Docker.
- Orquestación mediante Kubernetes.
- Observabilidad mediante Prometheus.
- Persistencia especializada utilizando Neo4j.

4. Guías de operación y mantenimiento

Operación del Sistema

Construcción y ejecución de pruebas

Antes de realizar cualquier despliegue se debe verificar que el proyecto compile correctamente y que todas las pruebas se ejecuten sin errores.

```
.\gradlew.bat test
```

```
.\gradlew.bat build
```

Resultado esperado: BUILD SUCCESSFUL

Gestión de contenedores Docker

Para generar la imagen de un microservicio:

```
docker build -t circleguard-auth-service .
```

```
docker images circleguard-auth-service
```

Despliegue en Kubernetes

El despliegue de los servicios se realiza mediante manifiestos Kubernetes.

```
kubectl apply -f k8s\
```

```
kubectl get pods -A
```

```
kubectl get deployments -A
```

Monitoreo

Los componentes de observabilidad se despliegan mediante:

```
kubectl apply -f k8s\observability\
```

```
kubectl -n observability get pods
```

Las métricas son recolectadas por Prometheus y visualizadas mediante Grafana.

Mantenimiento

Consulta de logs

```
kubectl logs <nombre-pod> --tail=100
```

Reinicio controlado

```
kubectl rollout restart deployment/circleguard-auth-service
```

```
kubectl rollout status deployment/circleguard-auth-service
```

Rollback

Ante una falla durante el despliegue:

kubectl rollout undo deployment/circleguard-auth-service -n circleguard-master

Incidentes frecuentes

Incidente	Accion Correctiva
Pod en CrashLoopBackOff	Revisar logs y variables de entorno
Error de conexión a base de datos	Verificar servicios y credenciales
Imagen Docker no encontrada	Validar nombre de imagen y registry
Servicio no disponible	Revisar deployments, services y probes

5. Análisis de Resultados de pruebas

Estrategia de Pruebas

El proyecto implementa pruebas unitarias, de integración, end-to-end y pruebas de rendimiento con el objetivo de validar la estabilidad de los microservicios y la correcta interacción entre los componentes de la arquitectura.

Resultados Obtenidos

La ejecución de la suite de pruebas produjo los siguientes resultados:

Indicador	Resultado
Total de pruebas ejecutadas	112
Fallos	0
Errores	0
Pruebas omitidas	7
Estado general	Exitoso

Resultado de Gradle: BUILD SUCCESSFUL

Pruebas Unitarias

Validaron la lógica de negocio de servicios, controladores, repositorios, listeners y convertidores.

Pruebas de Integración

Verificaron la comunicación entre componentes como:

- Auth Service e Identity Service.
- Dashboard Service y la base de datos.
- Form Service y Kafka.
- File Service y Auth Service.
- Pruebas End-to-End

Permitieron validar flujos completos de autenticación, gestión de identidad, formularios, dashboard y manejo de archivos.

Pruebas de Rendimiento

Se utilizaron scripts desarrollados en Locust para simular carga sobre los servicios.

Las métricas evaluadas fueron:

- Tiempo promedio de respuesta.
- Percentil 95 (P95).
- Percentil 99 (P99).
- Requests por segundo.
- Tasa de errores.

Los scripts fueron validados correctamente y se encuentran preparados para ejecutarse sobre ambientes desplegados.

Conclusiones

Los resultados obtenidos evidencian una plataforma estable, sin errores de compilación ni fallos en las pruebas automatizadas ejecutadas durante la validación del proyecto.

6. Documentación de Infraestructura como Código y Release Notes

Infraestructura como Código (IaC)

La infraestructura del proyecto fue definida mediante Terraform utilizando una arquitectura modular y reutilizable.

Estructura

```
infra/
├── terraform/
│   ├── modules/
│   │   ├── namespace/
│   │   ├── microservice/
│   │   └── observability/
│   └── envs/
│       ├── dev/
│       └── stage/
```

Ambientes

- Development (Dev)
- Staging (Stage)
- Production (Prod)

Beneficios

- Reproducibilidad de infraestructura.
- Control de versiones.
- Automatización de despliegues.
- Reducción de errores manuales.

Costos estimados de infraestructura

Componente	Dev	Stage	Prod
Kubernetes	Bajo	Medio	Alto
PostgreSQL	Bajo	Medio	Alto
Neo4j	Bajo	Medio	Alto
Kafka	Medio	Medio	Alto
Observabilidad	Bajo	Medio	Medio/Alto
CI/CD y Registry	Bajo	Medio	Medio

Para entornos académicos se recomienda Docker Desktop o Minikube. Para producción se recomienda utilizar servicios administrados en la nube.

Release Notes

Se implementó un mecanismo automatizado para la generación de notas de versión a partir del historial Git.

Comando utilizado:

```
mkdir -p build/release-notes git log --pretty=format:'- %h %s (%an)' > build/release-notes/release.md
```

Versionamiento

Se adoptó versionamiento semántico mediante etiquetas:

vMAJOR.MINOR.PATCH

Ejemplos de versiones

- v1.0.0 – Versión inicial de la plataforma.
- v1.1.0 – Integración de Kubernetes, Jenkins y Terraform.
- v1.1.1 – Correcciones y mejoras de estabilidad.

Cada despliegue incorpora evidencia de pruebas, imagen desplegada y procedimiento de rollback para garantizar la trazabilidad de los cambios.